

Research Overview:

The research for this project focused on understanding how real-world nurseries operate, the factors affecting plant growth, and the management systems used to monitor and maintain plant health. Key research areas included plant classification, growth requirements, and nursery management practices.

A nursery is a controlled environment where various plants are cultivated for commercial or research purposes. Effective nursery management involves organizing plants based on their characteristics such as size, type, and environmental needs. Each plant category requires specific care routines — including water regulation, temperature control, and exposure to sunlight — to ensure optimal growth.

Four main categories of plants were researched for this project:

Tropical Plants: Require warm temperatures, high humidity, and consistent watering. Examples include *Bird of Paradise* and *Pitcher Plants*.

Carnivorous Plants: Adapted to nutrient-poor soils, these plants rely on trapping insects for nutrients. Examples include *Sundew* and *Nepenthes*.

Succulent Plants: Store water in their leaves and thrive in dry environments. Examples include *Aloe Vera* and *Hen and Chick*.

Temperate Plants: Grow in moderate climates and often have seasonal growth patterns. Examples include *Lilac* and *White Oak*.

This classification directly informed the GreenHouse design in our simulation, where each plant type is managed under its own factory, ensuring accurate representation of environmental needs.

Influence on Design Decisions:

The design of our GreenHouse simulation is driven using multiple **object-oriented design patterns** that collectively ensure modularity, flexibility, and realism. Each pattern was carefully selected to solve a specific design challenge encountered in simulating a functioning plant nursery. For instance, creational patterns such as **Abstract Factory** and **Prototype** streamline plant creation by ensuring that new plant types can be easily instantiated or cloned without modifying existing code. Meanwhile, **Structural patterns** like **Decorator** and **Composite** enhance scalability by allowing dynamic addition of features (such as pots or fertilizer) and by efficiently organizing customers and company groups within the sales subsystem.

On the behavioural side, patterns such as **State**, **Observer**, **Command**, and **Chain of Responsibility** bring the simulation to life by modelling plant growth, health monitoring, and

automated care. The **Iterator** pattern ensures seamless traversal of the plant inventory stored in a BST, while the **Mediator** facilitates communication between staff, plants, and customers, reducing tight coupling. Finally, the **Memento** pattern provides reliability by enabling stock or command recovery through system snapshots. Together, these patterns create a highly interactive and autonomous simulation, mirroring the real-world operations of a nursery while maintaining code reusability, scalability, and ease of maintenance.

Design Pattern Application

This section details the design patterns implemented in the Greenhouse Nursery Simulation, explaining their purpose, implementation, and how they collectively enhance system flexibility, modularity, and maintainability.

Prototype Pattern

Category: Creational

Purpose in Our Design:

Enables cloning of existing Plant objects to efficiently handle bulk orders or duplicate plants with similar attributes without recreating them from scratch.

Implementation Summary:

The Plant base class defines a clone() method that allows derived classes to return copies of themselves.

When multiple plants of the same type are requested, the system replicates prototypes rather than reinitializing each one, optimizing performance.

Abstract Factory Pattern

Category: Creational

Purpose in Our Design:

Provides an interface for creating related plant objects without specifying their exact classes, ensuring consistent plant creation across various greenhouse environments.

Implementation Summary:

>The GreenHouse class acts as the **Abstract Factory**, defining methods for creating plants of different categories.

>Factories such as TropicalPlantFactory, CarnivorousPlantFactory, SucculentPlantFactory, and

>TemperatePlantFactory each implement plant creation specific to their climate.

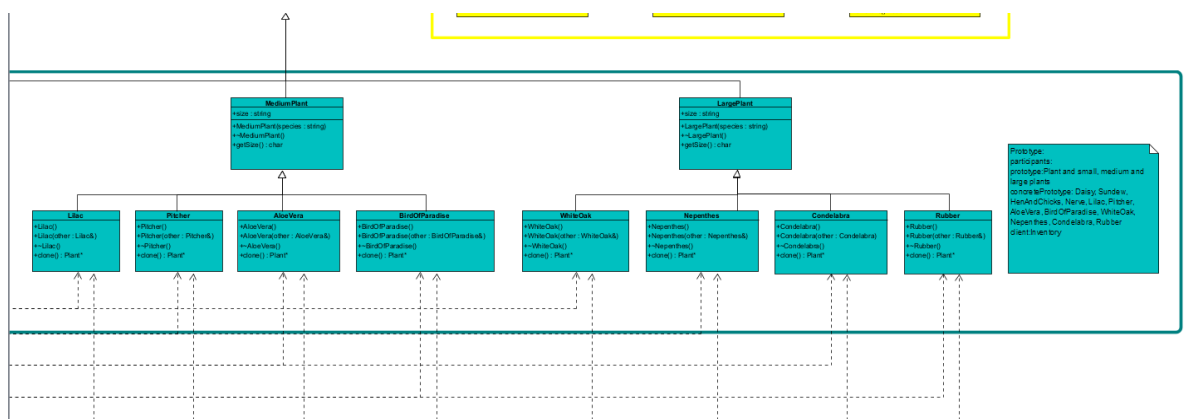
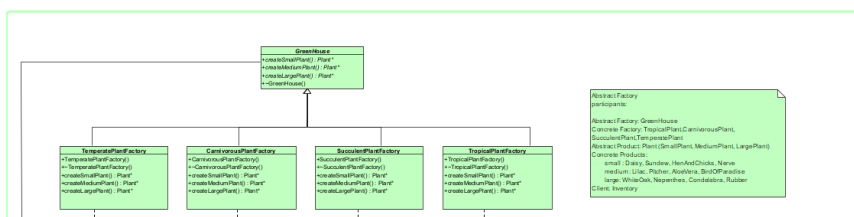
Abstract products (Plant) and concrete subclasses (SmallPlant, MediumPlant, LargePlant) represent plant categories, ensuring consistent characteristics across all product families.

```

class Plant {
    Species : string
    WaterLevel : int
    SunlightLevel : int
    FertilizerLevel : int
    PruneLevel : int
    IGrowthStage : int
    <CareStrategy : PlantCareHandler*
    growthState : GrowthState*
    healthState : HealthState*
    climate : string
    description : string
    price : double
    size : string
    currentCycleCount : int
    seedCyclesNeeded : int
    sproutCyclesNeeded : int
    matureCyclesNeeded : int
    observer : ConcreteGrowthObserver*
    readyForStock : bool

    +Plant()
    +Plant(species : string)
    +Plant(const other : Plant&)
    +~Plant()
    +getPrice() : double
    +getDescription() : string
    +getClimate() : string
    +clone() : Plant*
    +setCareStrategy(strategy : PlantCareHandler*) : void
    +getGrowthState() : GrowthState*
    +setGrowthState(state : GrowthState*) : void
    +grow() : void
    +getHealthState() : HealthState*
    +setHealthState(state : HealthState*) : void
    +attach(observer : ConcreteGrowthObserver*) : void
    +detach() : void
    +notify() : void
    +getSpecies() : string
    +setPrice(newPrice : double) : void
    +setDescription(newDesc : string) : void
    +isReadyForStock() : bool
    +markReadyForStock() : void
    +getWaterLevel() : int const
    +setWaterLevel() : void
    +getSunlightLevel() : int const
    +setSunlightLevel() : void
    +getFertilizerLevel() : int const
    +setFertilizerLevel() : void
    +getPruneLevel() : int const
    +setPruneLevel() : void
    +completeCareSession() : void
    +getIsize() : string const
    +getCurrentCycleCount() : int const
    +getSeedCyclesNeeded() : int const
    +getSproutCyclesNeeded() : int const
    +getMatureCyclesNeeded() : int const
    +resetCycleCount() : void
    +printCurrentNeeds() : void
    +printGrowthStatus() : void
    +printHealthStatus() : void
    +printFullStatus() : void
    +shouldBeRemovedFromInventory() : bool
}

```



Decorator Pattern

Category: Structural

Purpose in Our Design:

Adds extra features to Plant objects dynamically without modifying their original class.

Implementation Summary:

PotDecorator and FertilizerDecorator extend the base Plant functionality.

PotDecorator adds structural modifications (e.g., decorative pots), while FertilizerDecorator provides behavioural enhancements that influence plant growth.

>This ensures flexible feature addition without subclass explosion.



Composite Pattern

Category: Structural

Purpose in Our Design:

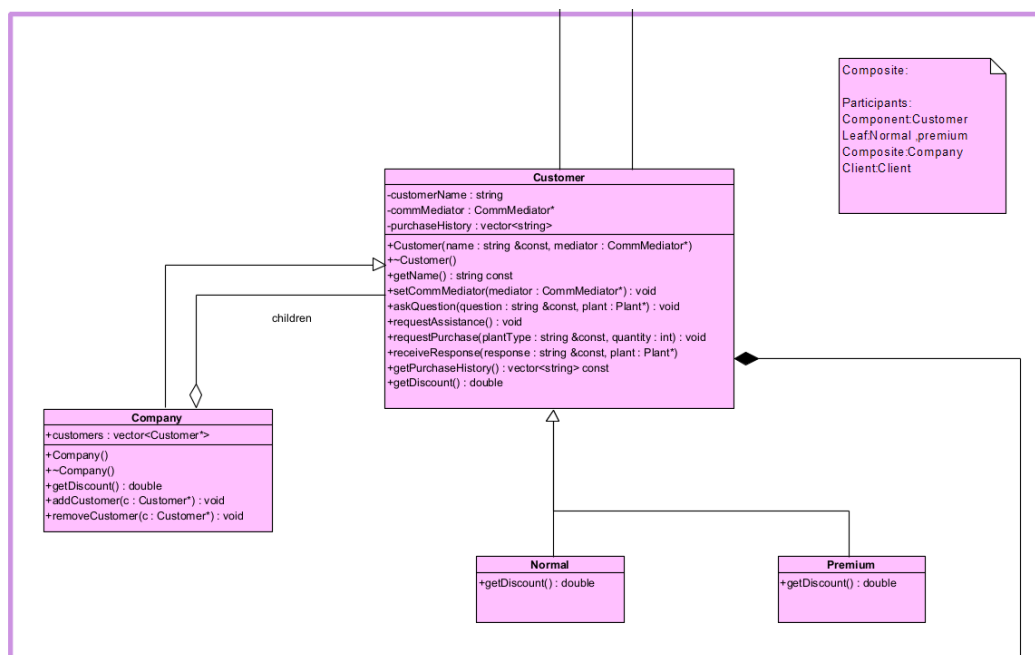
Organizes customers into a hierarchy of individuals and company clients, allowing both to be treated uniformly.

Implementation Summary:

>Customer is the base component class.

>Individual Customer and Company Customer extend it, where a company can hold multiple individuals as sub-customers.

>This simplifies group orders, transactions, and exclusive discounts for companies.



Iterator Pattern

Category: Behavioural

Purpose in Our Design:

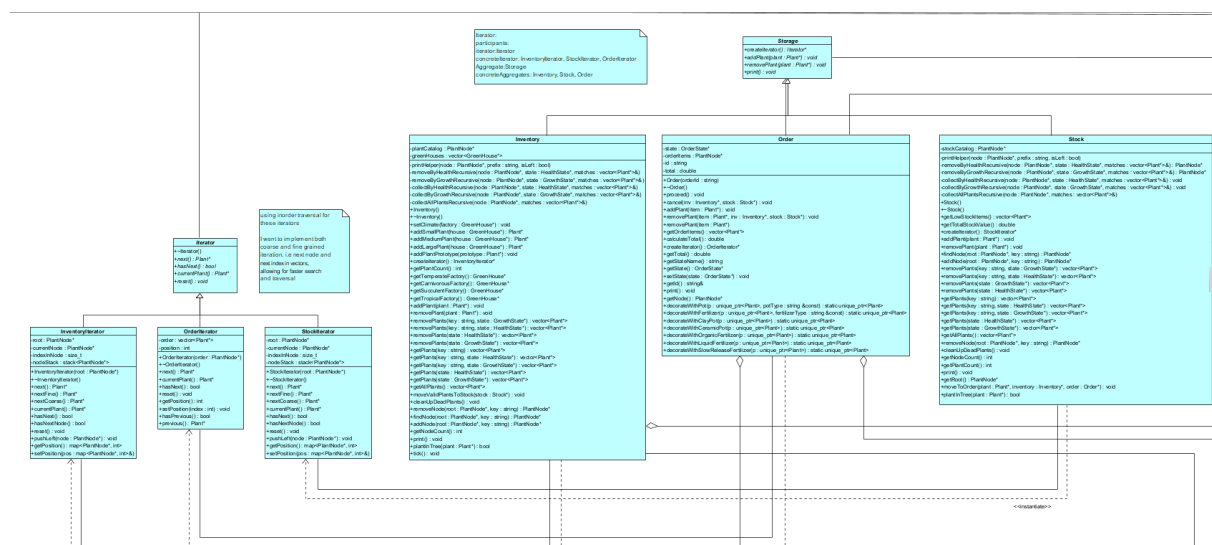
Facilitates traversal through the nursery's inventory system without exposing internal data structures.

Implementation Summary:

>The **Inventory** is implemented as a **Binary Search Tree (BST)** where each node represents a plant species.

>Each node stores a vector of Plant pointers.

>The PlantIterator class provides in-order traversal of BST nodes and their internal vectors, enabling alphabetical iteration of plant species.



Observer Pattern

Category: Behavioural

Purpose in Our Design:

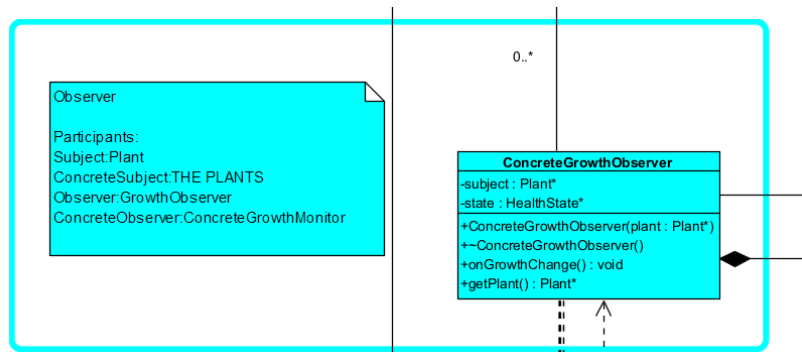
Monitors changes in plant health and notifies staff when care is required.

Implementation Summary:

>Each Plant object attaches a GrowthObserver to monitor its health.

>When a plant's health drops below 60%, the observer alerts a StaffMember, triggering the necessary care actions.

>Promotes a one-to-many relationship, ensuring scalability across multiple plants.



Command Pattern

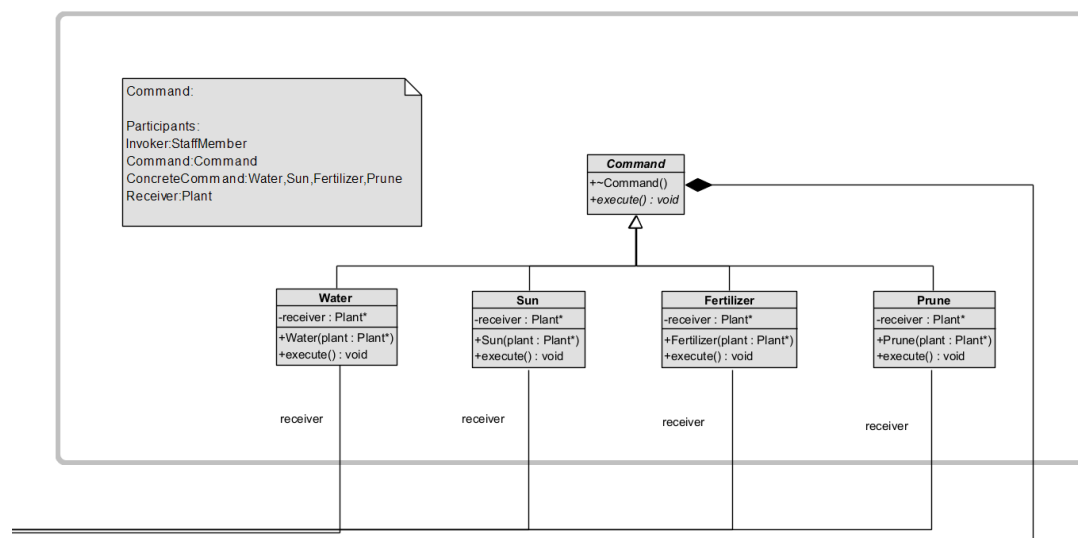
Category: Behavioural

Purpose in Our Design:

Encapsulates plant care actions as command objects that staff members execute to restore plant health.

Implementation Summary:

- >Commands include WaterCommand, SunCommand, FertilizerCommand, and PruneCommand.
- >StaffMember acts as the **Invoker**, executing commands based on observer notifications.
- >Each command modifies plant attributes and can be stored or replayed for consistency.



Chain of Responsibility Pattern

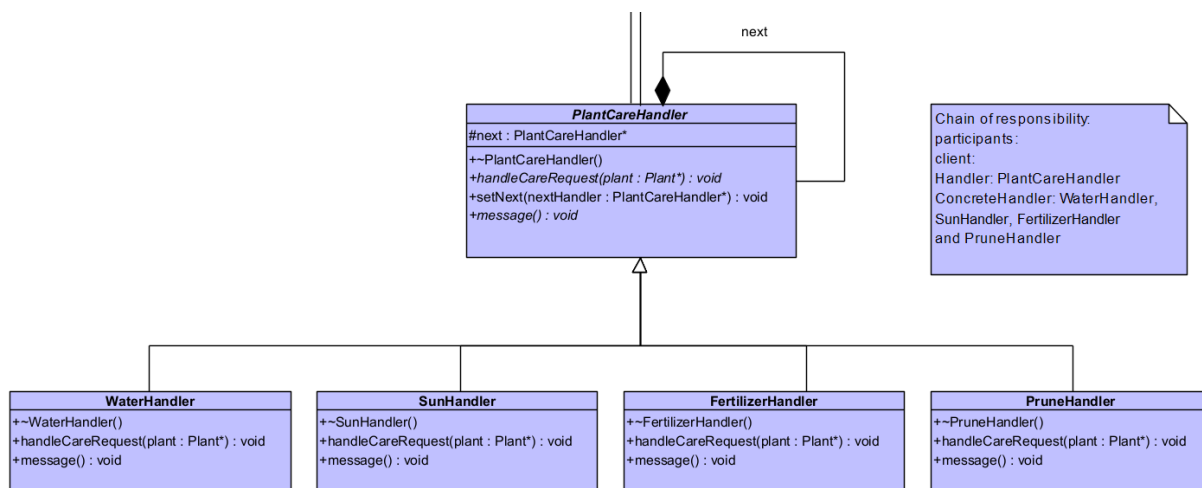
Category: Behavioural

Purpose in Our Design:

Links multiple plant-care handlers to automatically determine and respond to specific plant needs.

Implementation Summary:

- >The `tick()` function in `Plant` gradually decreases care levels.
- >When levels drop below 60%, the `GrowthObserver` triggers the **Chain of Responsibility**.
- >Handlers like `WaterHandler`, `SunlightHandler`, and `FertilizerHandler` check requirements in order and invoke the proper **Command**.
- >This decouples detection from the action, improving maintainability.



Mediator Pattern

Category: Behavioural

Purpose in Our Design:

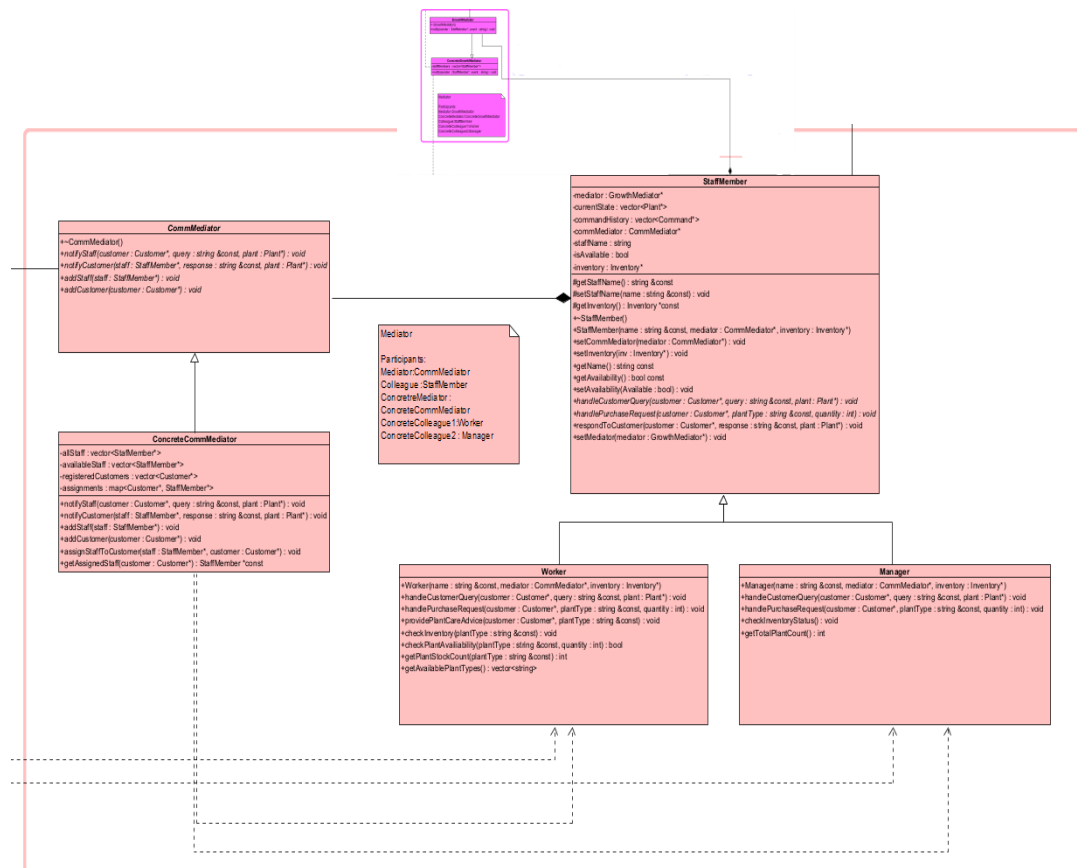
Coordinates communication between objects, reducing direct dependencies.

Implementation Summary:

>GrowthMediator facilitates communication between Plant and StaffMember.

>CommMediator manages communication between Customer and StaffMember.

>Simplifies object interaction, ensuring modularity and reducing system coupling.



Memento Pattern

Category: Behavioural

Purpose in Our Design:

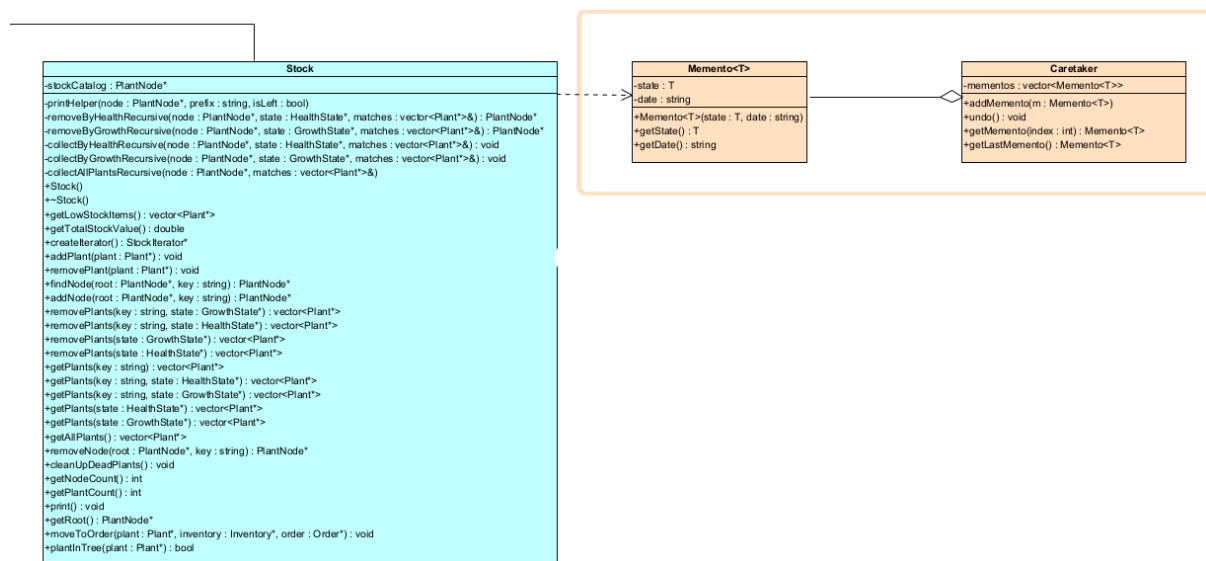
Enables snapshot saving and restoration of stock states, providing an undo mechanism.

Implementation Summary:

>Integrated with the inventory subsystem to store system states.

>Before large operations or stock updates, a Memento captures the current inventory snapshot.

>Allows rollback to previous states to recover from accidental modifications.



State Pattern

Category: Behavioural

Purpose:

Manages various plant and order states — including growth, health, and purchase stages — through encapsulated state transitions.

Implementation Summary:

>GrowthState, HealthState, and OrderState manage lifecycle and transaction flow.

>Simplifies complex state transitions by keeping behaviour within individual state classes.

>Supports autonomous plant lifecycle management and smoother process control.

